

Независимый от поставщика контракт ModelClient

Основной план, модуль 01

Краткое содержание

`ModelClient` отделяет код приложения от протокола конкретного поставщика и выполняет ровно одно обращение к языковой модели. `ModelRequest` описывает намерение приложения, `ModelResponse` — завершённый результат, а `ModelEvent` — одно типизированное событие потокового ответа. Общий контракт должен нормализовать смысл, необходимый вызывающему коду, но одновременно сохранять исходные блоки, идентификаторы, порядок, причины остановки и сведения об использовании. Потеря неизвестного поля опаснее его временного непонимания. Практика закрепляет эту границу неизменяемыми моделями, детерминированной имитацией и проверяемым жизненным циклом потока; инженерная защита связывает честное отсутствие идентификатора поставщика с восстановлением состояния агента после неоднозначного побочного эффекта.

Ключевые понятия

- **ModelClient** — асинхронная граница одного обращения: конкретная реализация адаптера преобразует запрос и нормализует ответ либо ошибку. Не выбирает маршрут, не поддерживает цель агента и не выполняет вызов инструмента.
- **ModelRequest** — выражает `request_id`, `model`, `messages`, доступные инструменты в поле `tools` и требуемый формат ответа. Не копирует HTTP-запрос одного API и не описывает политику маршрутизации.
- **ModelResponse** — хранит упорядоченные результаты завершённого обращения: текст, структурированные данные, вызов инструмента, отказ и сведения об использовании. Не является только строкой.
- **ModelEvent** — представляет смысловой шаг потока: начало ответа, фрагмент, завершение результата, сведения об использовании или завершённый ответ. Частичный JSON ещё нельзя исполнять.
- **ProviderPayload** — хранит точные байты поставщика вместе с `provider`, `kind` и медиа-типом. Неизвестные расширения доступны для воспроизведения и аудита, но не получают полномочий агента.

Основные идеи

Проблема: одинаковый смысл, разные формы

OpenAI Responses возвращает упорядоченные элементы `output`, Anthropic Messages — блоки `content` и `stop_reason`, Gemini `generateContent` — `candidates` с `content.parts`,

`finishReason` и `usageMetadata`. Во всех трёх системах может появиться текст, структурированный результат, вызов инструмента, отказ или ограничение длины. Но совпадение смысла не означает совпадения данных.

Если приложение читает каждую форму напрямую, протокол поставщика проникает в цикл агента, тесты и бизнес-правила. Если адаптер оставляет только строку, он теряет `call_id`, данные безопасности, цитаты, элементы рассуждения, `thoughtSignature`, подробности кэша и новые типы блоков. Нужны одновременно общая форма и доступ к исходным данным без переупаковки с потерями.

Один поток выполнения

```
приложение
-> ModelRequest
-> ModelClient
-> адаптер поставщика -> API поставщика
<- исходный ответ или исходные события потока
<- ModelResponse или поток ModelEvent + сохранённые данные поставщика
```

Для обычного вызова адаптер разбирает весь ответ и возвращает `ModelResponse`. Для потока он выдаёт `ModelEvent` в исходном порядке; сборка всех событий должна приводить к тому же смысловому `ModelResponse`, что и непотоковый вызов. Если поток оборвался, частичный результат остаётся наблюдаемым, но не становится успешным завершённым ответом.

Что приводить к общей форме

Общий смысл	Нормализованное представление
Текст	Упорядоченные текстовые блоки, а не только склеенная строка
Структурированный ответ	Исходный JSON-текст, проверенное значение и идентификатор применённой схемы
Вызов инструмента	Необязательный идентификатор вызова поставщика, имя, согласованные исходные и разобранные аргументы и порядок; выполнение остаётся снаружи <code>ModelClient</code>
Сведения об использовании	Общие <code>input</code> , <code>output</code> , <code>total</code> , если поставщик их сообщил; отсутствие остаётся отсутствием
Результат генерации	<code>completed</code> , <code>tool_requested</code> , <code>refused</code> , <code>truncated</code> , <code>blocked</code> или <code>incomplete</code>
Поток	Начало, части блоков, завершение блоков, сведения об использовании и завершающее событие
Ошибка границы	Общая категория с исходной причиной, статусом, идентификатором запроса и <code>retry_after</code> , если они есть

Строка, похожая на JSON, не становится структурированным ответом сама по себе. Для этого `ModelRequest` должен потребовать схему, а адаптер — проверить результат по этой схеме.

Что сохранять без потерь

- имя поставщика, фактически модель и версия, идентификаторы ответа и запроса;
- исходные блоки в исходном порядке вместе с неизвестными вариантами;
- специфичные для поставщика причина остановки, состояние и детали отказа;
- цитаты, оценки безопасности и обратную связь по инструкции;
- элементы рассуждения, зашифрованное содержимое, непрозрачные отпечатки и связь с вызывающим кодом;
- Gemini `thoughtSignature` и порядок частей, необходимый для продолжения;
- подробные поля учёта использования: чтение и запись кэша, рассуждение, использование серверных инструментов;
- HTTP-статус, заголовки `request-id / retry-after` и исходное тело ошибки;
- типы и данные неизвестных потоковых событий.

«Без потерь» означает сохранить полученные байты либо структурное дерево без отбрасывания полей. Преобразовать неизвестный блок в `dict`, удалить поля и затем сериализовать обратно — уже не обратимое преобразование без потерь.

Примеры

Примеры сокращены до полей, важных для сравнения; это не полные ответы API.

1. OpenAI Responses: `function_call`

```
{
  "id": "resp_oai_1",
  "status": "completed",
  "output": [{
    "type": "function_call", "call_id": "call_1",
    "name": "get_weather", "arguments": "{\"city\":\"Казань\"}"
  }],
  "usage": {"input_tokens": 42, "output_tokens": 18}
}
```

Общая форма получает вызов инструмента с `call_id` поставщика, именем и аргументами. Наличие идентификатора зависит от поставщика; общий контракт не должен выдумывать его. Соответствие исходных и разобранных аргументов проверяется отдельно от проверки `ToolSpec.input_schema`. Нельзя оставить только вспомогательное свойство набора разработчика `output_text`: OpenAI Responses использует массив типизированных элементов, где рядом могут находиться блоки `message`, `reasoning` и другие. Исходный элемент сохраняется целиком.

2. Anthropic Messages: отказ как результат

```
{
  "id": "msg_ant_1", "type": "message",
  "content": [{"type": "text", "text": "Не могу выполнить запрос."}],
  "stop_reason": "refusal",
  "stop_details": {"type": "refusal", "category": "policy"},
  "usage": {"input_tokens": 31, "output_tokens": 9}
}
```

Это успешный HTTP-обмен с завершённым результатом `refused`, а не `ModelTransportError`. Общая форма сохраняет видимый текст, но `stop_reason` и `stop_details` нужны для политики резервного маршрута и аудита. В потоке эти сведения появляются позднее текста в `message_delta`.

3. Gemini `generateContent`: структурированный текст и метаданные

```
{
  "candidates": [{
    "content": {"role": "model", "parts": [
      {"text": "{$\"umbrella\":true}", "thoughtSignature": "opaque..."}
    ]},
    "finishReason": "STOP", "safetyRatings": []
  ]},
  "usageMetadata": {"promptTokenCount": 24, "candidatesTokenCount": 7},
  "modelVersion": "provider-version", "responseId": "gem_1"
}
```

После проверки запрошенной схемы JSON можно представить как структурированный результат. `thoughtSignature`, `safetyRatings`, `modelVersion`, `responseId` и исходный `Part` нельзя выбрасывать. Если `promptFeedback.blockReason` присутствует и `candidates` отсутствуют, это нормальный заблокированный результат, а не ошибка разбора массива.

Потоковые различия

- OpenAI Responses выдаёт типизированные события SSE, например `response.created`, `response.output_text.delta`, `response.function_call_arguments.delta` и `response.completed`.
- Anthropic Messages выдаёт `message_start`, жизненный цикл каждого блока `content`, `message_delta` и `message_stop`; `input_json_delta.partial_json` нельзя разбирать как завершённый JSON до `content_block_stop`.
- Gemini `streamGenerateContent` возвращает последовательность `GenerateContentResponse`; новый Interactions API использует события `step.start`, `step.delta`, `step.stop` и `interaction.completed`.

`ModelEvent` должен выражать общий смысл, но всегда нести исходный тип и данные события. Неизвестный тип передаётся как неизвестное событие поставщика, а не молча игнорируется.

Реализованный механизм

Практика реализована без набора разработчика и реального API:

- `contracts.py` задаёт неизменяемые `ModelRequest`, `ModelResponse`, варианты результатов и событий, `Usage`, `ProviderPayload`, ошибки и протокол `ModelClient`;
- `scripted_client.py` потребляет ровно один сценарий на вызов, сверяет `request_id` и `schema_id`, вызывает переданный валидатор структурированного ответа, не делает скрытых повторов и не выполняет вызов инструмента;
- `stream_assembly.py` проверяет последовательность, идентичность поставщика, жизненный цикл результата, согласованность фрагментов с итогом, сведения об использовании и единственный завершённый ответ;
- `contract_suite.py` фиксирует общий контракт поведения, который позже должен запускаться без изменения проверок для каждой реализации адаптера.

Текущий набор модуля содержит 67 детерминированных тестов. Они подтверждают глубокую неизменяемость JSON, различие `usage=None` и сообщённого нуля, сохранение исходных байтов, проверку структурированного ответа, отсутствие скрытого повтора, передачу отмены без преобразования и точный наблюдаемый префикс при `ModelStreamInterrupted`. Один `ScriptedModelClient` ещё не доказывает корректность преобразования данных реальных наборов разработчика.

Заметки и наблюдения

Классификация ошибок

- **`UnsupportedCapability`** — требуемая возможность отсутствует до запроса; не повторять на том же маршруте, сохранить требование.
- **`ModelAuthenticationError`** — неверный ключ API или недостаточно прав; не повторять без изменения конфигурации, сохранить статус и идентификатор запроса.
- **`ModelRequestRejected`** — некорректный запрос; не повторять без исправления, сохранить код ошибки поставщика и тело ответа.
- **`ModelRateLimited`** — ограничение частоты или квота; допустим только ограниченный повтор с `retry_after`, областью квоты и идентификатором запроса.
- **`ModelUnavailable`** — перегрузка или 5xx; допустим ограниченный повтор с общим лимитом, нужно сохранить статус и тип ошибки поставщика.
- **`ModelTimeout`** — истёк локальный `deadline`; повтор создаёт новое обращение и стоимость, поэтому нужно сохранить фазу запроса.
- **`ModelTransportError`** — сетевой сбой; сохранить исходное исключение и факт получения байтов.
- **`ModelProtocolError`** — неизвестная или недопустимая форма ответа; обычно не повторять через тот же адаптер, сохранить исходное тело или событие.
- **`ModelStreamInterrupted`** — поток оборван после событий; сохранить наблюдаемый префикс `ModelEvent` и признак неоднозначной полноты.

- **CancelledError** — внешняя отмена; передать без оборачивания и не запускать скрытый повтор.

`refused`, `truncated`, `blocked`, `tool_requested` и завершение по заданной последовательности токенов — результаты языковой модели, а не транспортные исключения. Так вызывающий код может отдельно решать, показать отказ, продолжить после инструмента, увеличить лимит или выбрать другую языковую модель.

Матрица контрактных тестов

Один набор поведения должен запускаться для каждой реализации адаптера и для детерминированной имитации. Сейчас проверки исполняются только для `ScriptedModelClient`; преобразования данных поставщиков ниже — цель будущих адаптеров, а не уже пройденный критерий темы.

Эталон контракта	Целевое преобразование данных поставщика	Проверенный сценарий имитации
Текст и порядок блоков сохранены	OpenAI <code>output</code> ; Anthropic <code>content[]</code> ; Gemini <code>parts[]</code>	Упорядоченный <code>ModelResponse</code> , включая <code>UnknownOutput</code>
Структурированный ответ проверен запрошенной схемой	Текстовый формат OpenAI; формат ответа Anthropic; схема ответа Gemini	Допустимый и недопустимый JSON, <code>schema_id</code> и переданный валидатор
Вызов инструмента сохраняет необязательный идентификатор, имя, исходные и разобранные аргументы	<code>function_call</code> ; <code>tool_use</code> ; <code>functionCall</code>	Заданный сценарием <code>ToolCallOutput</code> , включая <code>call_id=None</code>
Вызов инструмента не выполняется в <code>ModelClient</code>	Любой адаптер поставщика только нормализует предложение	У имитации отсутствует исполнитель
Сведения об использовании и специфичные для поставщика подробности не теряются	<code>usage</code> ; <code>usage</code> ; <code>usageMetadata</code>	<code>None</code> , сообщённый ноль и специфичные для поставщика счётчики
Отказ и ограничение длины остаются результатами	Состояние, блокировка и причина завершения поставщика	<code>ResponseOutcome</code> и соответствующий вариант результата
Поток собирается в тот же смысловой ответ	Типизированные события, жизненный цикл блока, части ответа	Один заданный ответ в обычной и потоковой форме
Частичный JSON не становится вызовом инструмента	Фрагмент аргументов или входного JSON	Оборванный префикс без <code>OutputCompleted</code>
Неизвестный блок или событие сохраняется	Новый тип данных поставщика не отбрасывается	Точные <code>kind</code> и байты
Идентичность ошибки и отмена сохраняются	Статус, идентификатор запроса, метаданные повтора и причина	Типизированные ошибки; исходный <code>CancelledError</code>

Контрактные тесты проверяют наблюдаемое поведение, а не классы набора разработчика. Иначе «общий» набор фактически закрепит реализацию первого адаптера.

Инженерная защита: сбой после `send_email`

Поставщик может корректно вернуть `ToolCallOutput(call_id=None)` . Это честное отсутствие идентификатора поставщика, а не повод генерировать значение внутри `ModelClient` . Для побочного эффекта приложение создаёт собственный стабильный `operation_id` , сохраняет

предложение и состояние до выполнения вызова инструмента, а после сбоя запускает детерминированное восстановление без нового решения языковой модели:

```
SENDING -- сбой после внешней попытки --> сохранено SENDING
перезапуск --> EFFECT_UNKNOWN --> сверка по operation_id
FOUND --> SUCCEEDED, повтор запрещён
TERMINALLY_ABSENT --> SENDING --> повтор --> SUCCEEDED
UNKNOWN --> EFFECT_UNKNOWN, автоматический повтор запрещён
```

`TERMINALLY_ABSENT` означает доказательство, что предыдущая попытка завершена и не была принята. Обычное «письмо пока не найдено» при согласованности в конечном счёте или незавершённом запросе — это `UNKNOWN`, иначе проверка `lookup` сама создаст дубликат. Восстановление принадлежит среде выполнения, а не языковой модели: предложение модели не является источником фактического состояния внешнего сервиса.

Эксперимент с отказом в `test_ambiguous_email_recovery.py` проверяет все три ветви:

```
uv run --frozen pytest -q \
  lessons/module_01_provider_contract/tests/test_ambiguous_email_recovery.py
```

Наблюдаемый результат — `3 passed: FOUND` не повторяет принятое письмо, `TERMINALLY_ABSENT` разрешает вторую попытку, а `UNKNOWN` оставляет агента в `EFFECT_UNKNOWN`. Цена решения — устойчиво сохранённое состояние, дополнительная проверка `lookup`, задержка и возможная ручная проверка. Для безвредных уведомлений предметная политика может предпочесть доставку хотя бы один раз и принять возможный дубликат.

Типичные ошибки

1. Свести ответ к `text: str` и потерять параллельные блоки, идентификаторы и цитаты.
2. Считать любой JSON-подобный текст структурированным ответом без запроса и проверки схемы.
3. Выполнять вызов инструмента в `ModelClient`, смешивая протокол и полномочия.
4. Разбирать аргументы вызова инструмента до завершающего события потока.
5. Превращать отсутствие `usage` в нули и тем самым искажать стоимость.
6. Считать отказ, блокировку по безопасности или предел токенов сетевой ошибкой.
7. Падать на новом блоке поставщика либо молча его отбрасывать.
8. Повторять внутри адаптера без общего лимита повторов и наблюдаемой причины.

Ограничения этой фазы

Тема содержит типизированные контракты Python, `ScriptedModelClient`, сборщик потока и детерминированные тесты, но не содержит сетевого адаптера, анализатора SSE, реального API, эксперимента с обратным давлением, передачи крайнего срока, маршрутизации или полноценного движка JSON Schema. Поля, которых нет у части поставщиков, остаются необязательными и не получают выдуманные значения. Одна имитация не доказывает

корректность преобразования данных набора разработчика и не закрывает критерий темы: общий набор ещё должны пройти две реальные реализации адаптера.

Модель восстановления показывает причинную границу только в тестах и не является промышленной подсистемой. Переданный идентификатор приложения не доказывает его генерацию, общее хранилище в памяти — лишь упрощённая замена сохраняемой транзакции, а `TERMINALLY_ABSENT` — строгое проверяемое условие, которого обычная папка отправленных сообщений может не предоставлять. Эксперимент не обещает гарантии ровно одного выполнения.

Вопросы для повторения

1. Почему `ModelResponse(text=...)` недостаточен даже для ответа, который визуально содержит только текст?
2. Где должен выполняться вызов инструмента и какую информацию обязан передать `ModelClient`?
3. Почему отсутствие `usage` нельзя нормализовать в нулевые токены?
4. Что должен сделать адаптер с новым неизвестным блоком `content`?
5. Как доказать тестом эквивалентность потокового и обычного ответа?
6. Почему `call_id=None` нельзя заменять случайным идентификатором поставщика и где должен жить принадлежащий приложению `operation_id`?
7. Почему «не найдено» не разрешает повтор, пока проверка `lookup` не доказала `TERMINALLY_ABSENT`?

Итоги

Общий контракт должен стабилизировать смысл, а не копировать JSON первого поставщика и не сводить все ответы к строке. `ModelRequest` выражает намерение, `ModelClient` владеет одним обменом, `ModelEvent` сохраняет причинный порядок потока, а `ModelResponse` представляет завершённый результат. Нормализованные поля делают цикл агента независимым от поставщика; сохранённые данные поставщика предотвращают потерю новых возможностей и диагностической информации. Практика показала дополнительную границу: идентификатор корреляции поставщика нельзя подменять идентификатором идемпотентности приложения. При неоднозначном побочном эффекте состояние восстанавливает детерминированная среда выполнения, а не языковая модель; автоматический повтор допустим только по явной предметной политике и проверяемому условию.

Источники

Внешние утверждения об API проверены по первичной документации `2026-07-31`; локальная реализация и тесты — `2026-08-01`:

- OpenAI Responses: сопоставление сообщений с типизированными элементами
- OpenAI Responses: типизированные события потока
- Спецификация OpenAI API, `POST /v1/responses` — OpenAPI `2.3.0`

- Anthropic Messages API: причины остановки
- Anthropic Messages API: жизненный цикл потока
- Версионирование Anthropic API — `anthropic-version: 2023-06-01`; могут добавляться новые необязательные поля, значения перечислений и типы событий
- Gemini `GenerateContentResponse`
- Версии Gemini API — Interactions API доступен в стабильной `v1`; набор разработчика по умолчанию использует `v1beta`
- Переход Gemini на Interactions API — `generateContent` считается устаревшим, но остаётся поддерживаемым
- Основной план: промпт 1.2
- Практика 1.2 и команды запуска
- Общий контракт поведения
- Эксперимент восстановления после отказа